

PPD - Laborator 5 - Documentatie

1. File Version

Rulare	Threads	Readers	Workers	Timp (ns)	Raport Ts/Tp
Secvențial	-	-	-	30206230	1
Paralel	6	4	2	43540610	0.693
Paralel	8	4	4	34879460	0.866
Paralel	12	4	8	29281730	1.03

2. File Version - 5000 de studenti

Rulare	Threads	Readers	Workers	Timp (ns)	Raport Ts/Tp
Secvențial	-	-	-	2376100470	1
Paralel	6	4	2	1571756310	1.51
Paralel	8	4	4	1214231290	1.95
Paralel	12	4	8	924998560	2.56

3. Database Version

Rulare	Threads	Readers	Workers	Timp (ns)	Raport Ts/Tp
Secvențial	-	-	-	33566640	1
Paralel	6	4	2	40844660	0.82
Paralel	8	4	4	36072080	0.93
Paralel	12	4	8	32960400	1.048

4. Arhitectura soluției și modelul de execuție

Soluția este organizată pe baza modelului **producător-consumator**, unde thread-urile de citire (producători) citesc perechi (*id*, *notă*) din mai multe surse (fișiere sau tabele din baza de date) și le introduc într-o coadă cu capacitate limitată. Pentru citire se utilizează un **thread pool**, fiecare task corespunzând citirii complete a unui fișier sau tabel.

Thread-urile *workers* (consumatori) extrag elemente din coadă și actualizează o listă înlănțuită comună. Coada este sincronizată cu ajutorul variabilelor condiționale, evitând atât depășirea capacității maxime, cât și așteptarea activă atunci când nu există elemente disponibile.

5. Sincronizare fine-grained asupra listei înlănțuite

Pentru operațiile asupra listei s-a utilizat **sincronizare fine-grained**, la nivel de nod, în locul sincronizării coarse-grained la nivel de listă. Fiecare nod conține un mutex propriu, care este blocat doar pe durata strict necesară operației curente.

Lista este implementată cu **noduri santinelă** (head și tail), ceea ce simplifică logica de inserare și elimină cazurile speciale. Metoda addOrUpdate permite agregarea notelor pentru același student într-un mod thread-safe, chiar și atunci când mai mulți workers procesează simultan perechi cu același id.

Această abordare reduce semnificativ conținutul între thread-uri și permite o scalare mai bună odată cu creșterea numărului de workers.

6. Construirea listei sortate în paralel

După finalizarea citirii și procesării tuturor datelor, worker-ii sincronizează execuția folosind o barieră. Ulterior, fiecare thread extrage noduri din lista inițială și le inserează într-o nouă listă, ListaSortata, utilizând metoda addInOrder.

Inserarea se face astfel încât lista rezultată să fie ordonată descrescător după notă (și crescător după id în caz de egalitate). Procesul este realizat în paralel, folosind aceleași mecanisme de sincronizare fine-grained, iar la final rezultatele sunt scrise în fișierul rezultat.txt.

7. Concluzii

Implementarea evidențiază avantajele utilizării sincronizării fine-grained și ale procesării paralele. Prin combinarea thread pool-ului pentru citire, a modelului producător-consumator și a listelor înlănțuite sincronizate la nivel de nod, soluția obține performanțe mai bune față de varianta secvențială, menținând în același timp corectitudinea datelor.